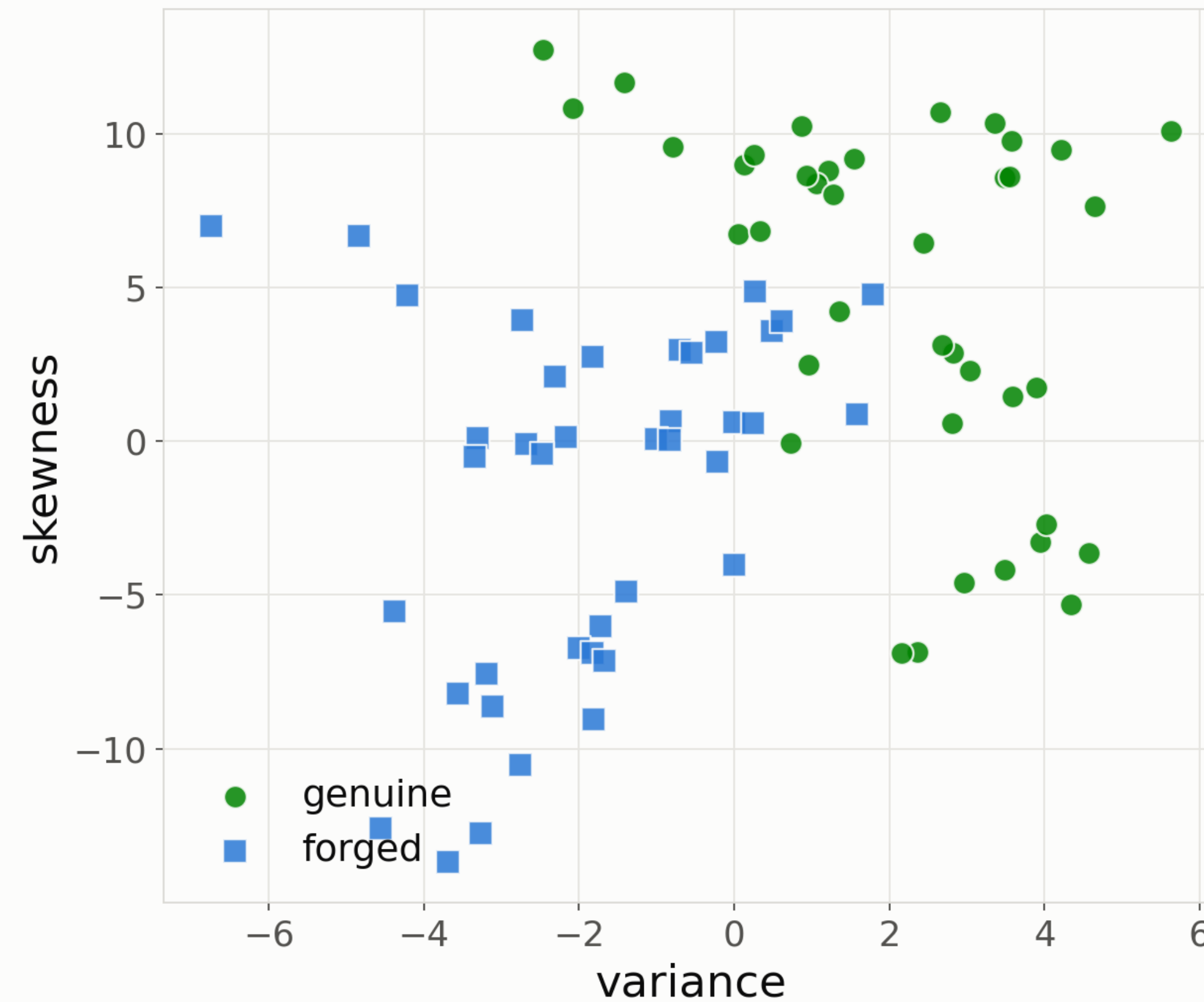


# Logistic regression



*10-701 Introduction to Machine Learning*  
*Geoff Gordon*

# *A more- interesting example*



*training set  
n=80 points*

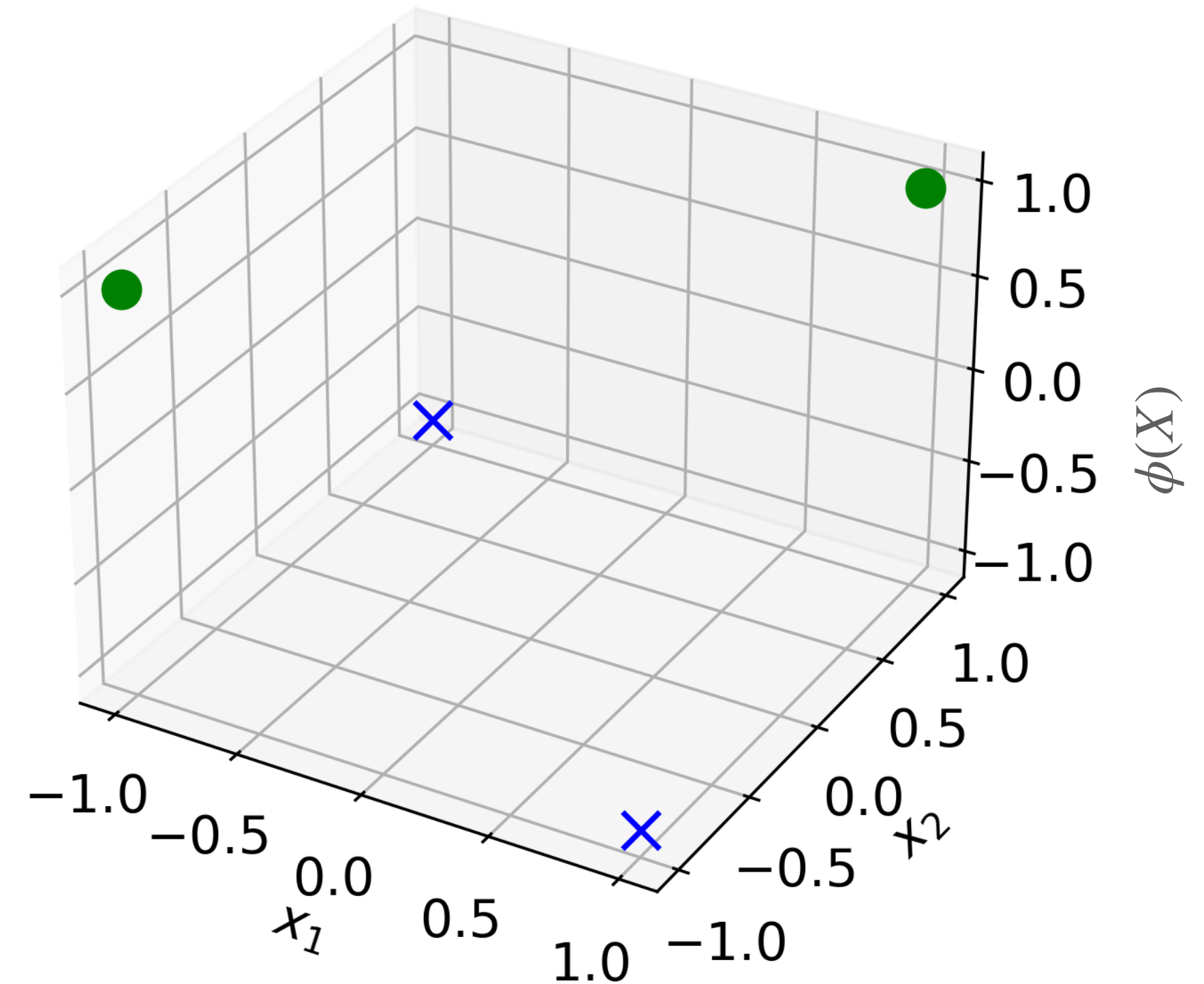
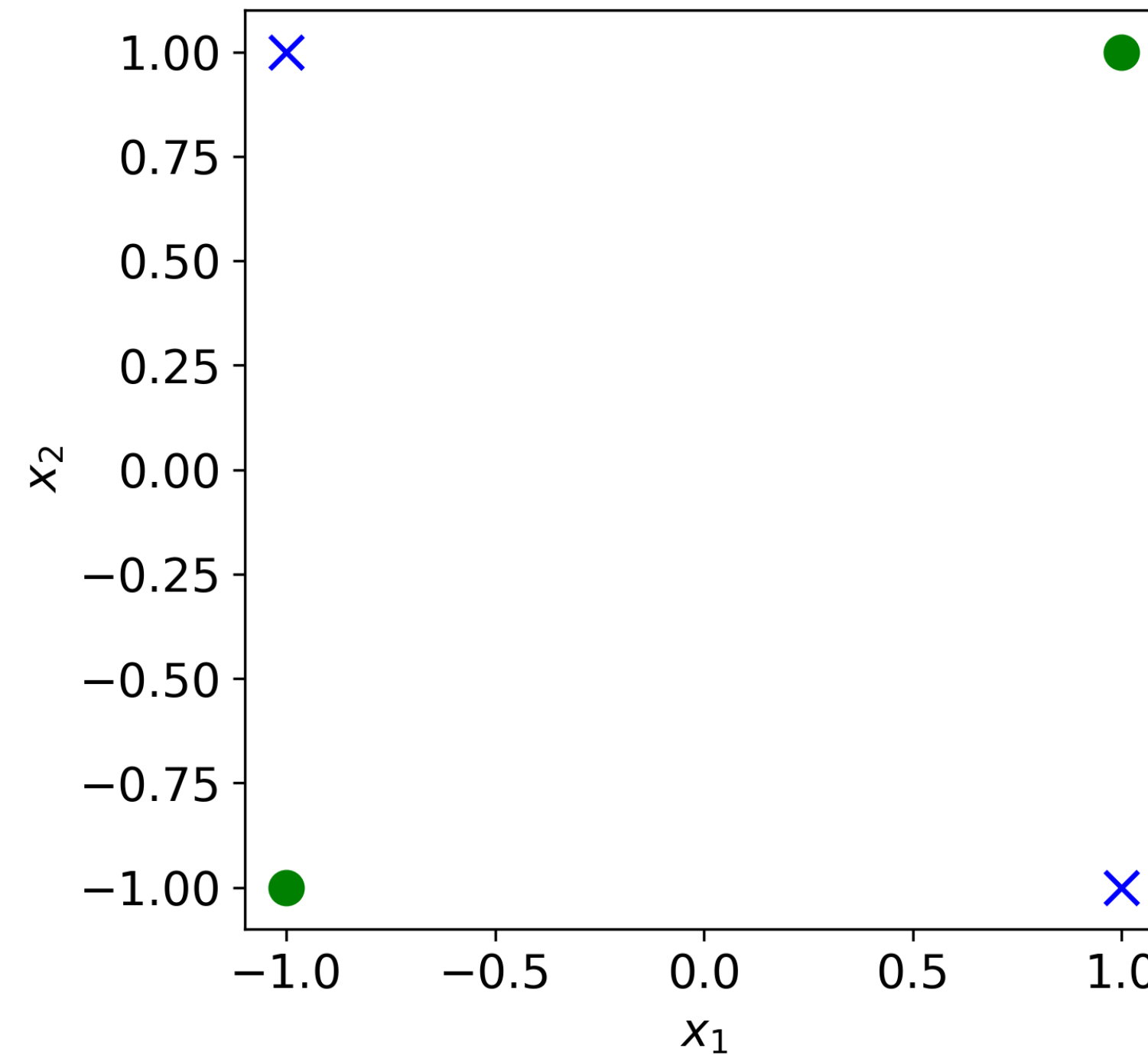
*full data has 4  
features, 1372  
examples*

- UCI banknote data: decide whether a banknote is genuine or forged by looking at statistics of a wavelet transform of scanned image
- Best decision boundary is not linear!

# Nonlinear decision boundaries with linear models

*boundary is linear in new higher-d feature space, but intersection with original feature space is nonlinear*

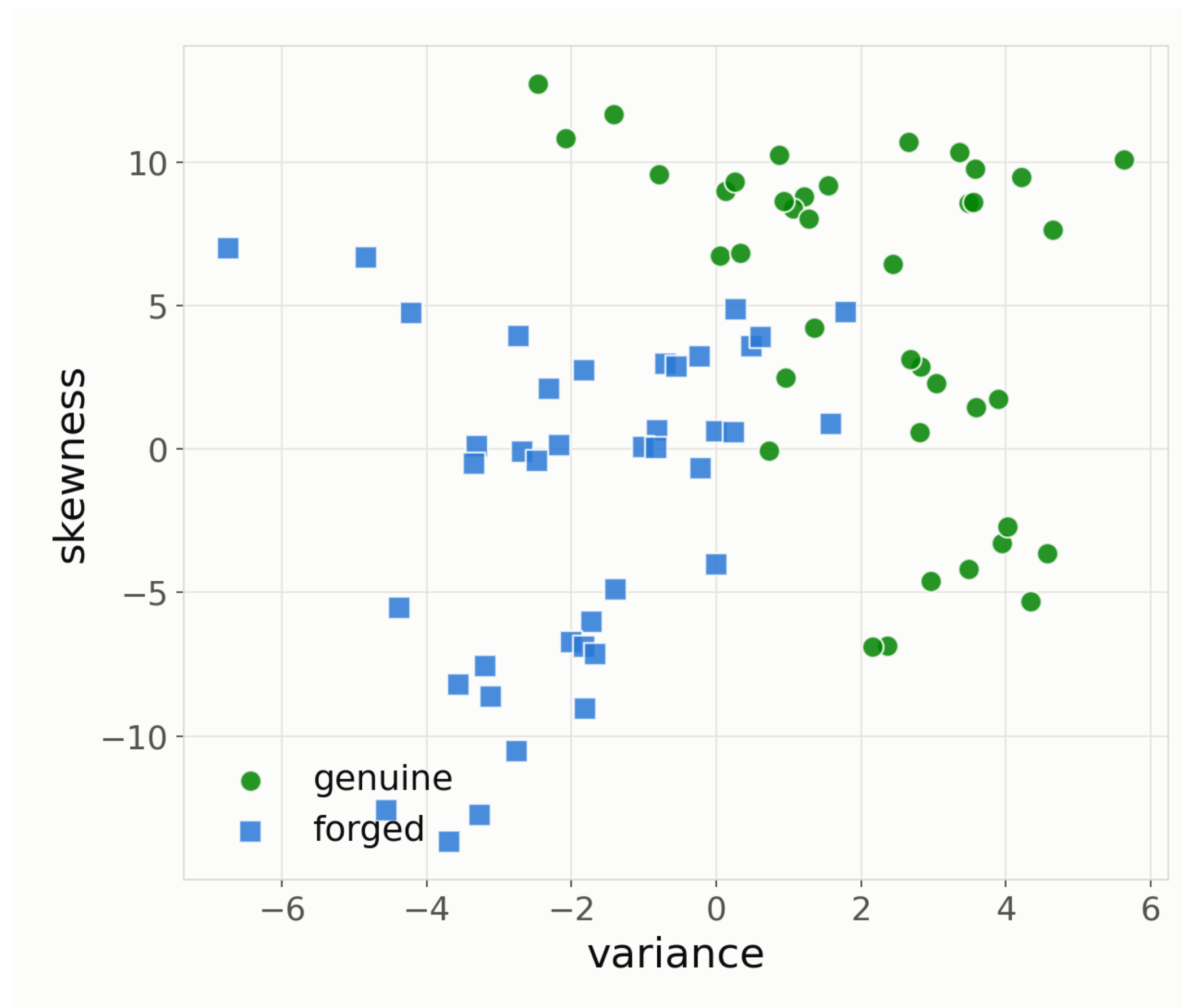
*and in both views, linear in parameters*



- Construct new features  $\phi_1(X), \phi_2(X), \dots$ : discriminant is  $w_1\phi_1(X) + w_2\phi_2(X) + \dots + b$
- E.g., above 2d data is not linearly separable, but if we add a third feature  $X_1X_2$ , we can separate it

$$\text{sgn}(0 \cdot X_1 + 0 \cdot X_2 + 1 \cdot X_1X_2 + 0)$$

# *How to add features to make it separable?*



*training set  
n=80 points*

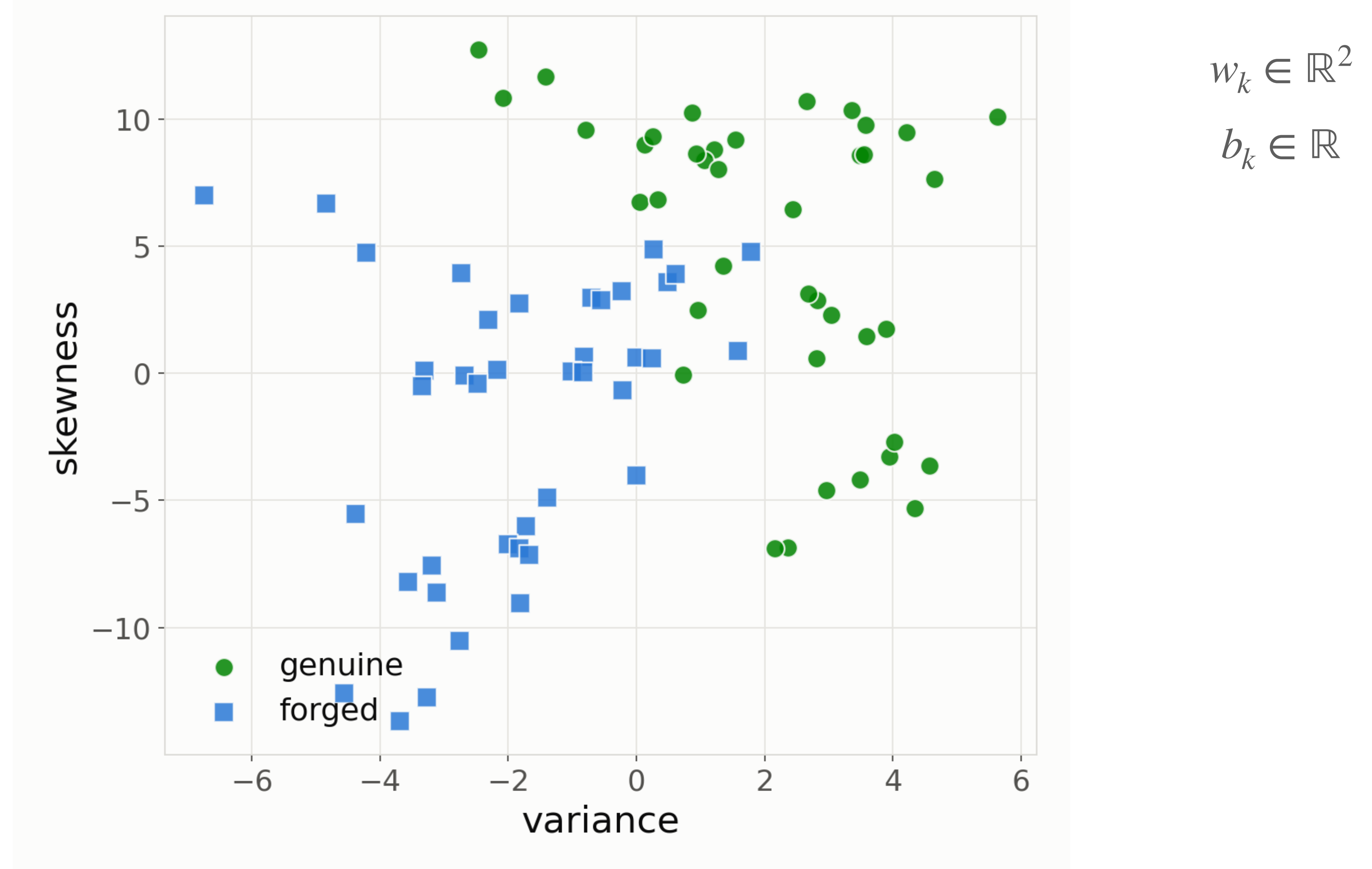
*full data has 4  
features, 1372  
examples*

- Problem: we don't know nearly as much about this data as we do about XOR, hard to use geometric intuition
- Need a general-purpose way to add nonlinear features

# Random features

*like a 2-layer neural network where we fix first-layer weights*

*a useful tool — some of the power of neural networks comes from random initialization*

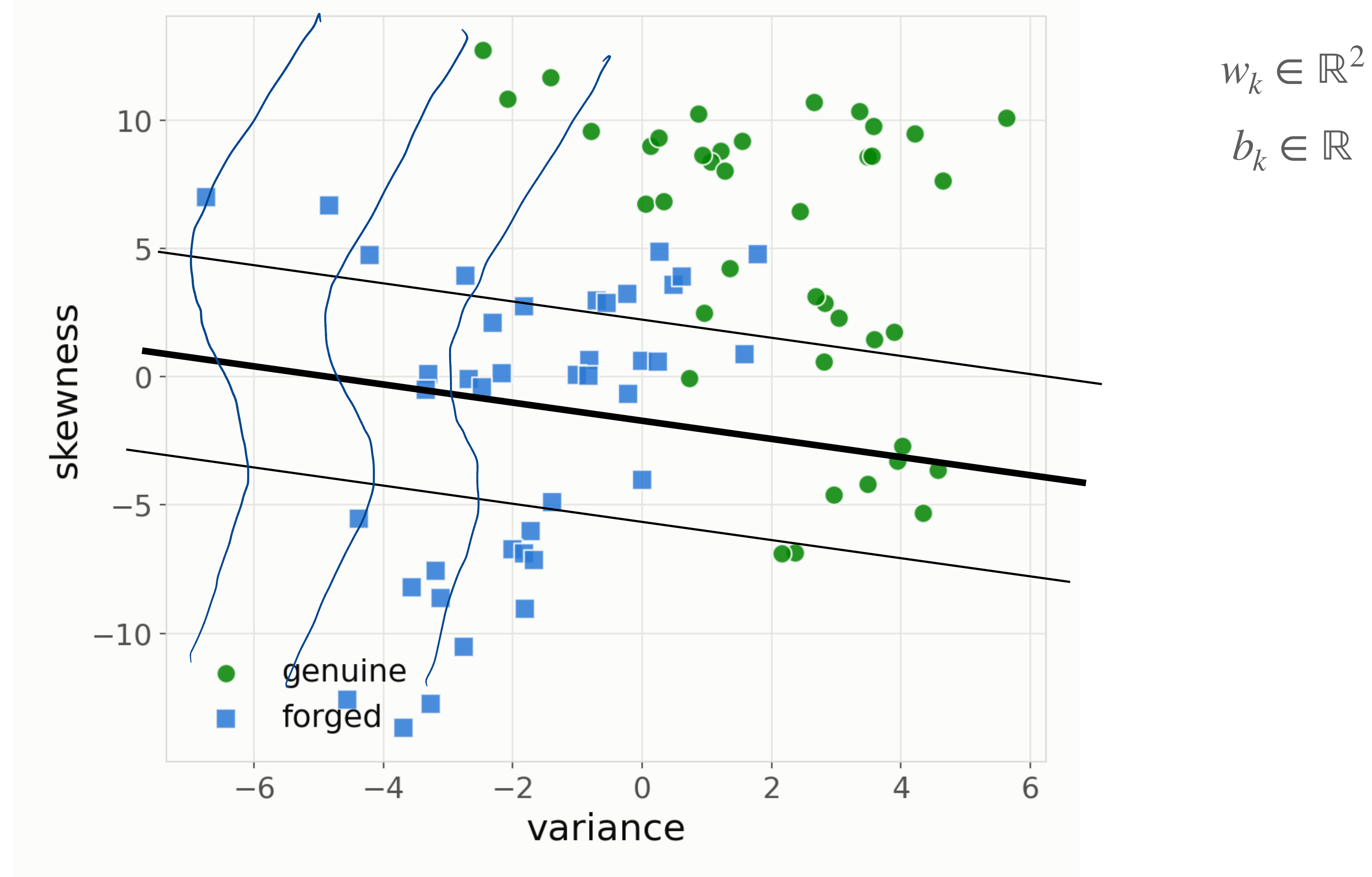


- Generate *random* features to try to make data separable
  - ▶  $\phi_k(X) = \sigma(w_k \cdot X + b_k)$ , variety of positions and slopes
  - ▶ more features = more expressive decision boundary

# Random features

*like a 2-layer neural network where we fix first-layer weights*

*a useful tool — some of the power of neural networks comes from random initialization*

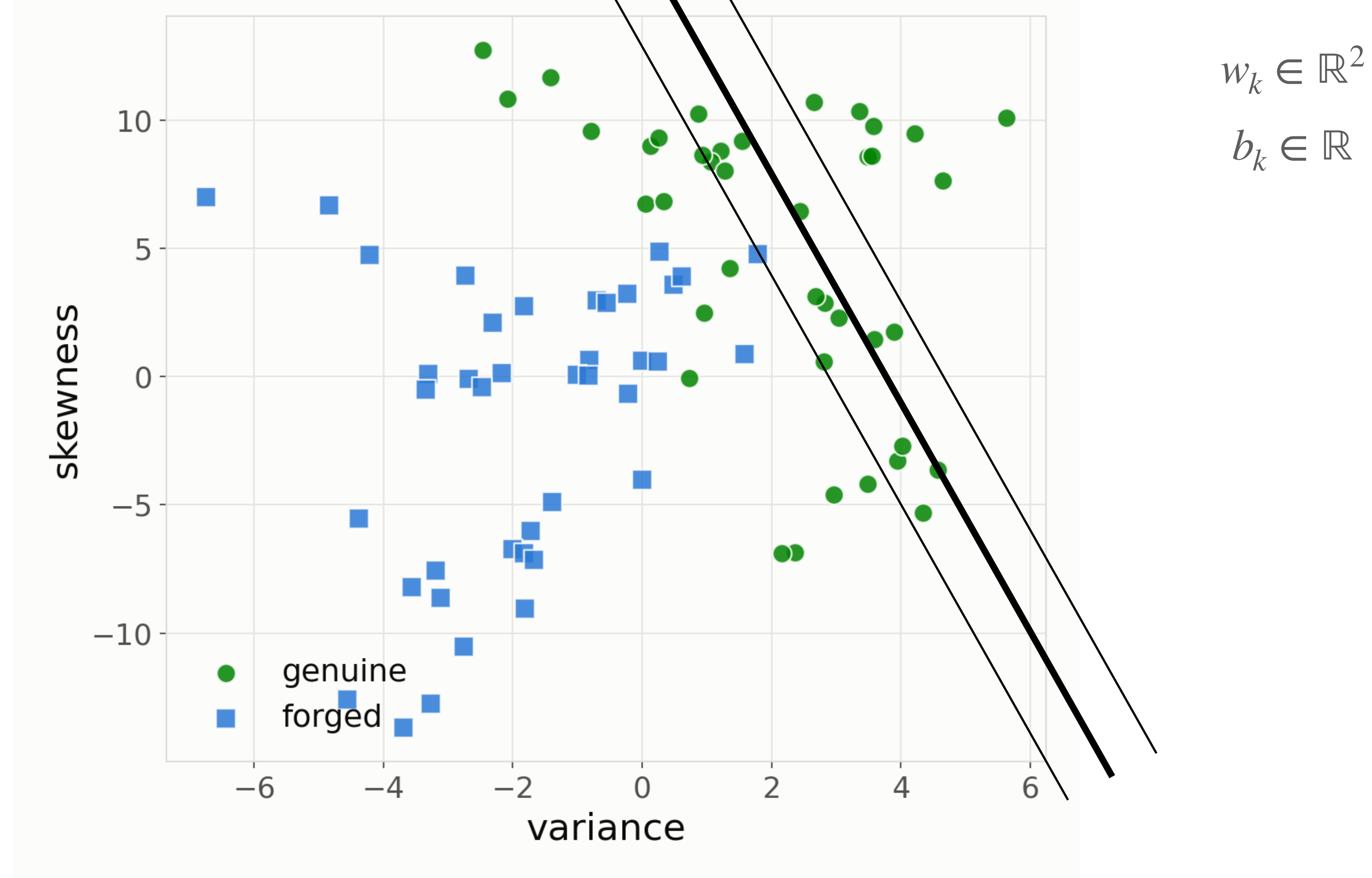


- Generate *random* features to try to make data separable
  - ▶  $\phi_k(X) = \sigma(w_k \cdot X + b_k)$ , variety of positions and slopes
  - ▶ more features = more expressive decision boundary

# Random features

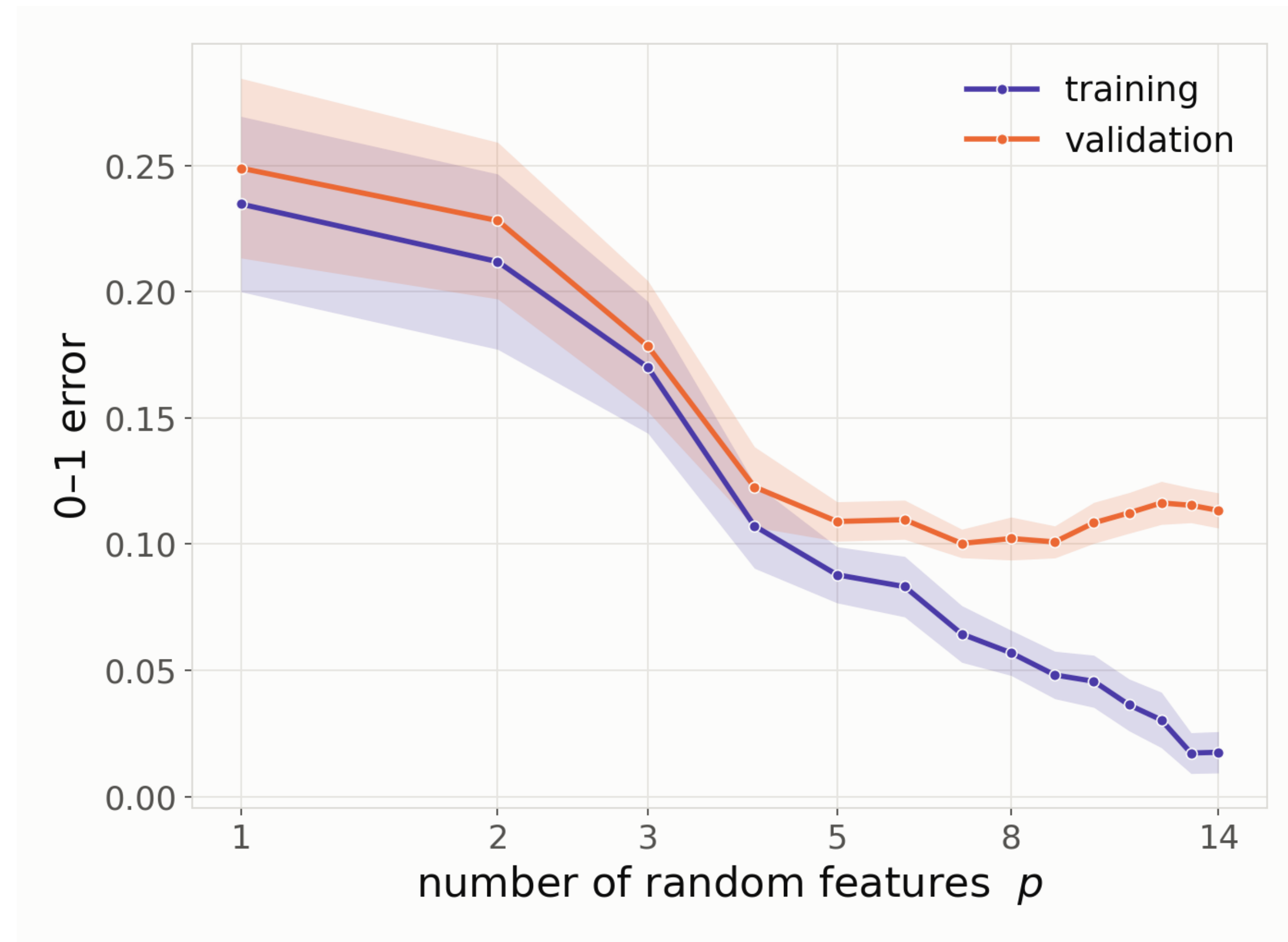
*like a 2-layer neural network where we fix first-layer weights*

*a useful tool — some of the power of neural networks comes from random initialization*



- Generate *random* features to try to make data separable
  - ▶  $\phi_k(X) = \sigma(w_k \cdot X + b_k)$ , variety of positions and slopes
  - ▶ more features = more expressive decision boundary

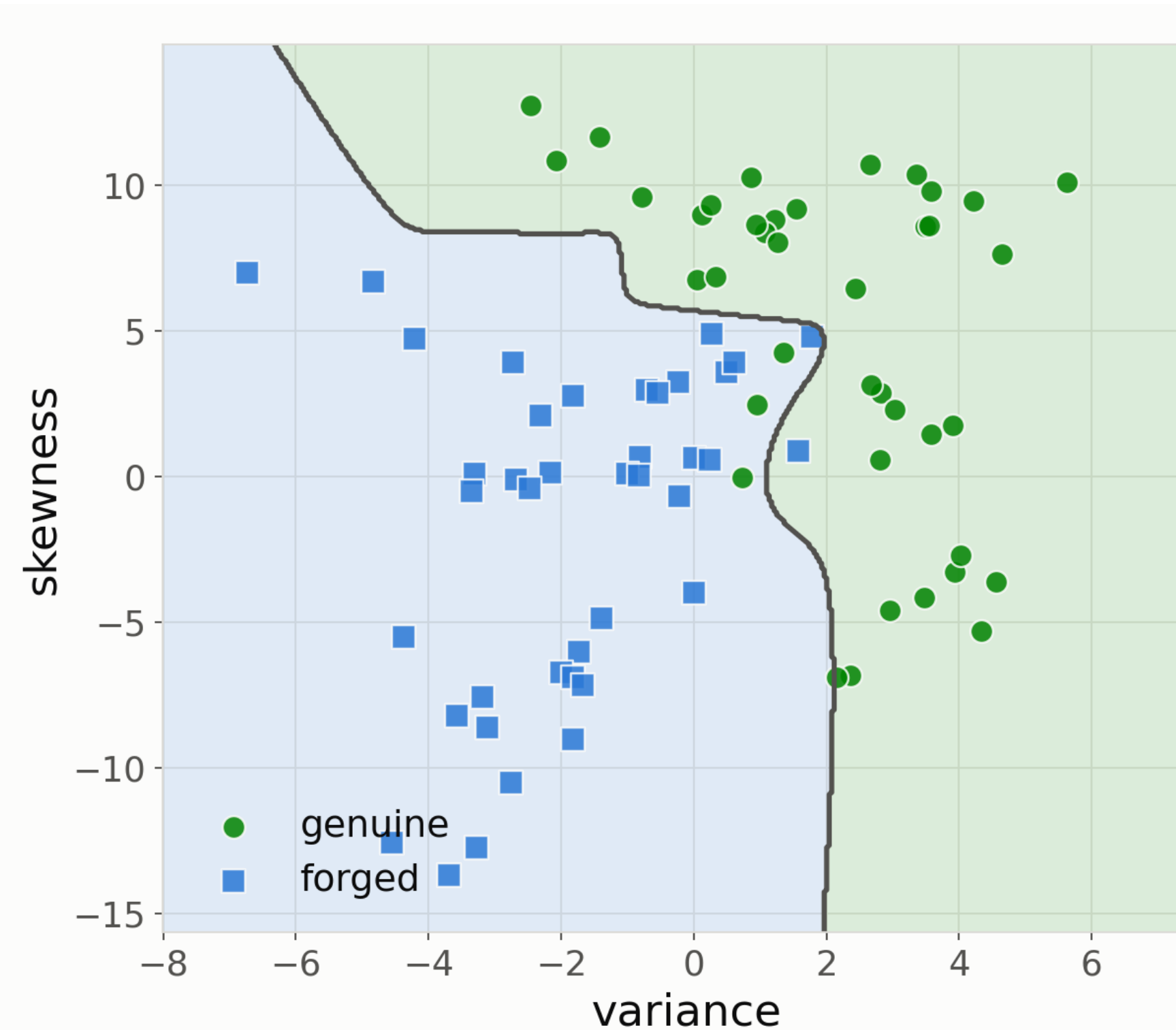
# Overfitting



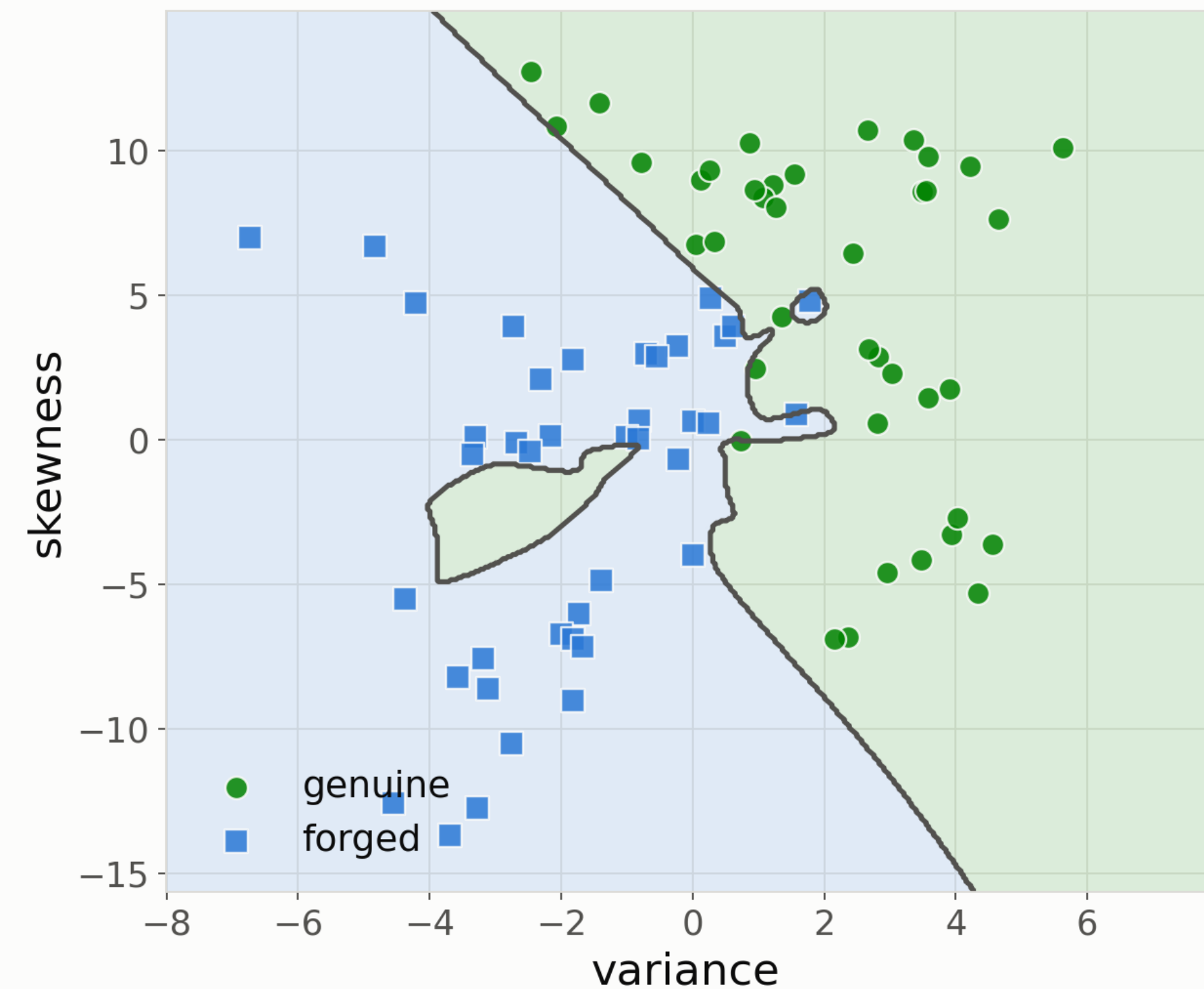
*mean  $\pm 1.96\sigma$   
over 40 seeds*

- As we increase the number of features, data does become more and more separable, but logistic regression overfits

# ***Decision boundaries***



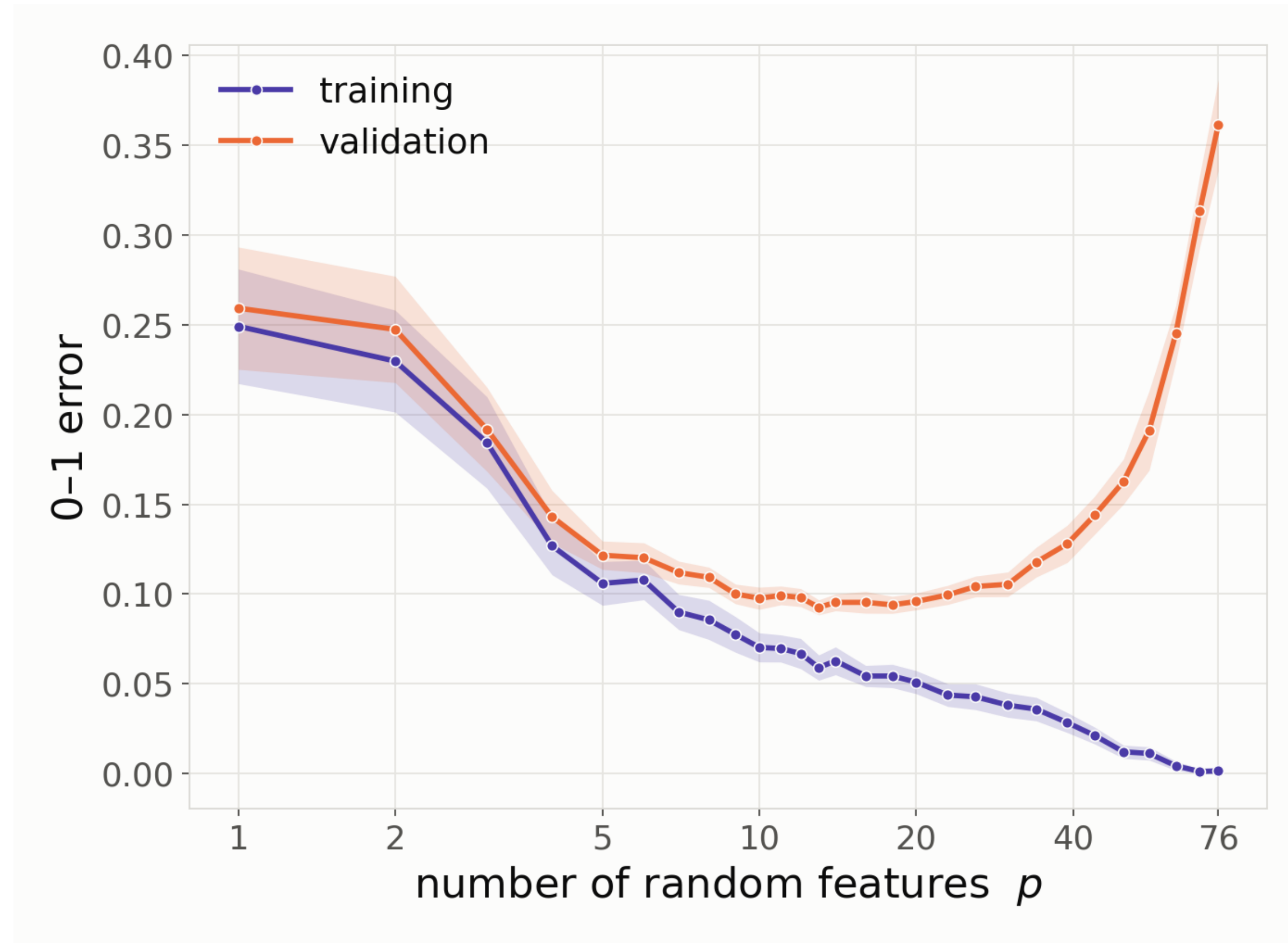
*9 features*



*18 features*

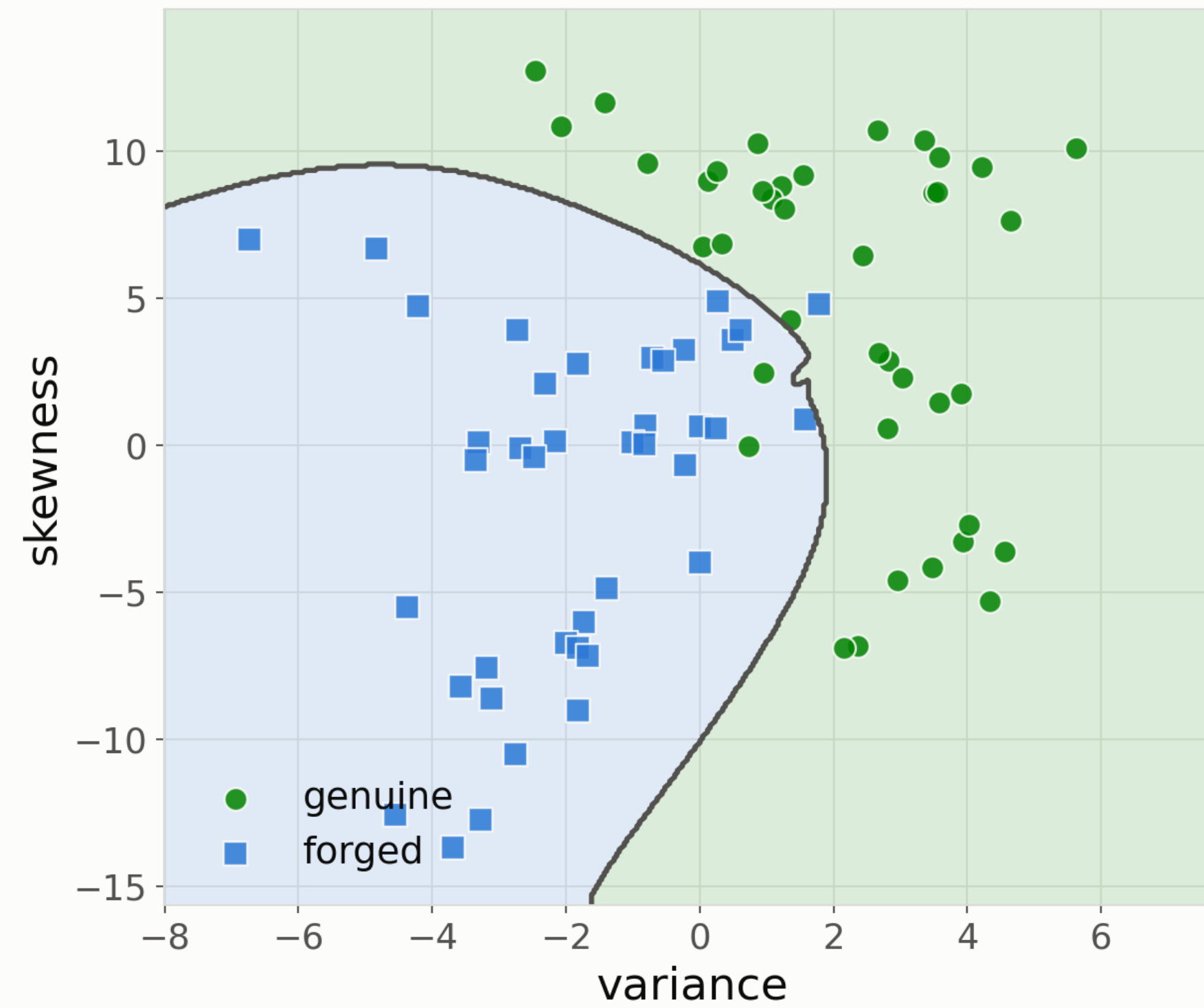
- Decision boundary gets more complex as we overfit

# Overfitting

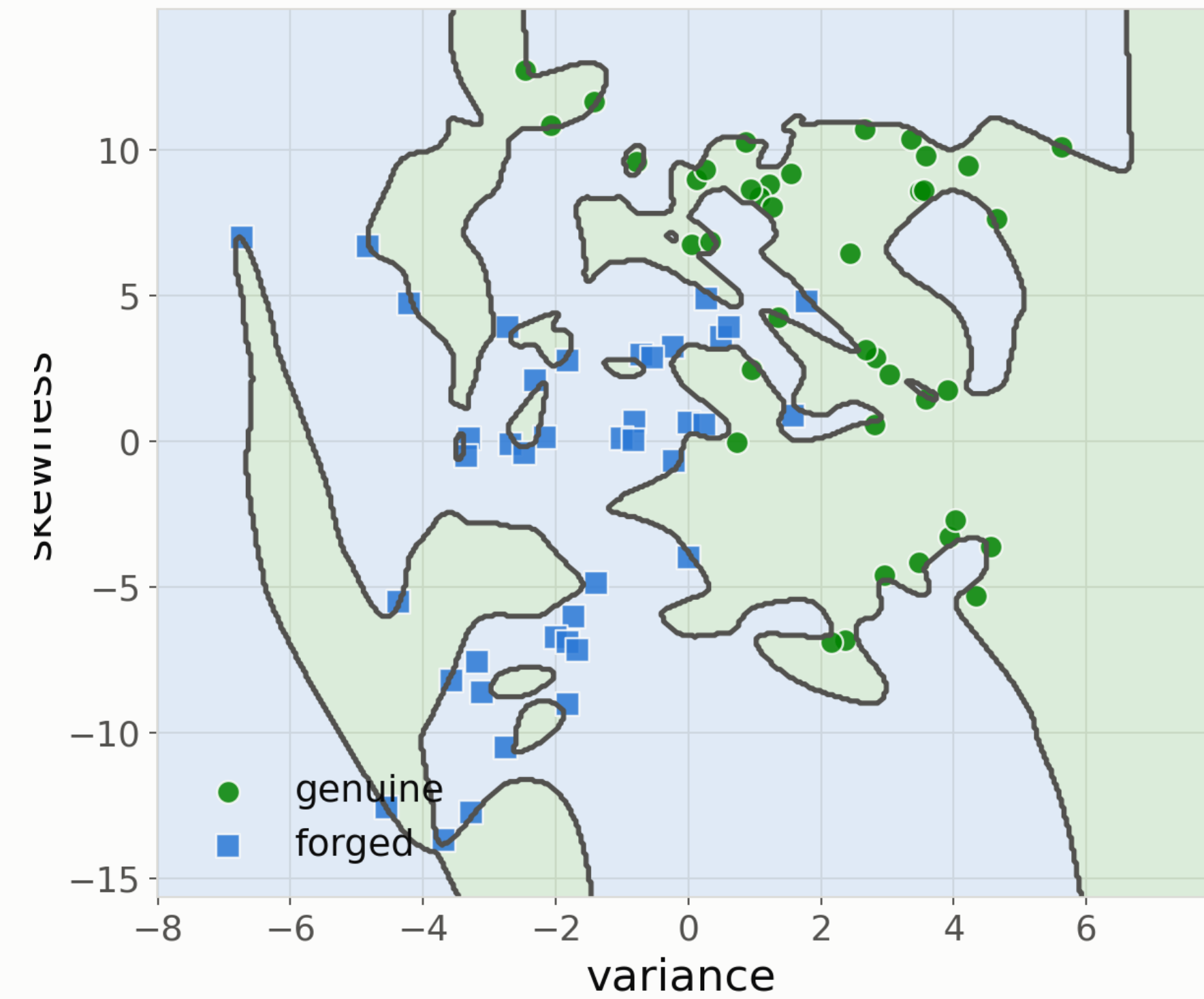


- Can run the same experiment with linear regression (minimize MSE instead of cross-entropy, targets still 0/1)
- Overfitting is worse: logistic regression benefits from its inductive bias towards high margin
- Also shifted right (regression needs more features)

# ***Decision boundaries***



*13 features*



*76 features*

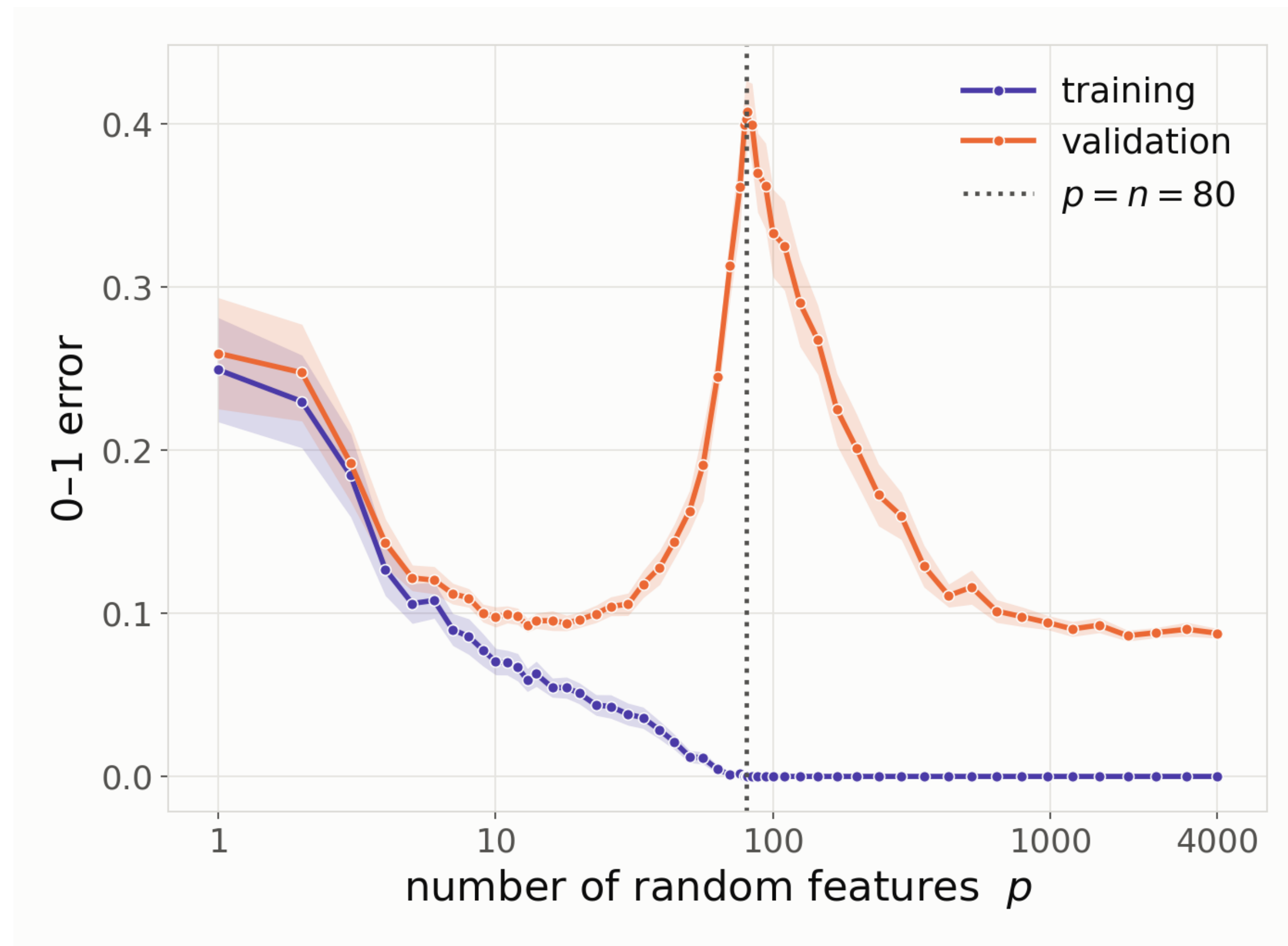
- Linear regression decision boundaries

## ***Preview: generalization in complex models***

- In the plots above, overfitting gets worse as the model class  $\mathcal{H}$  gets bigger / more complex
  - ▶ and classical overfitting bounds suggest this behavior: bound allows more overfitting as  $\mathcal{H}$  grows
- But more recently we've discovered cases where overfitting *goes back down* once  $\mathcal{H}$  passes a threshold!
  - ▶ classical bounds become loose, some other effect takes over — we have mixed understanding of these effects
- Simple example: ***double descent*** in linear models

# Double descent for linear regression

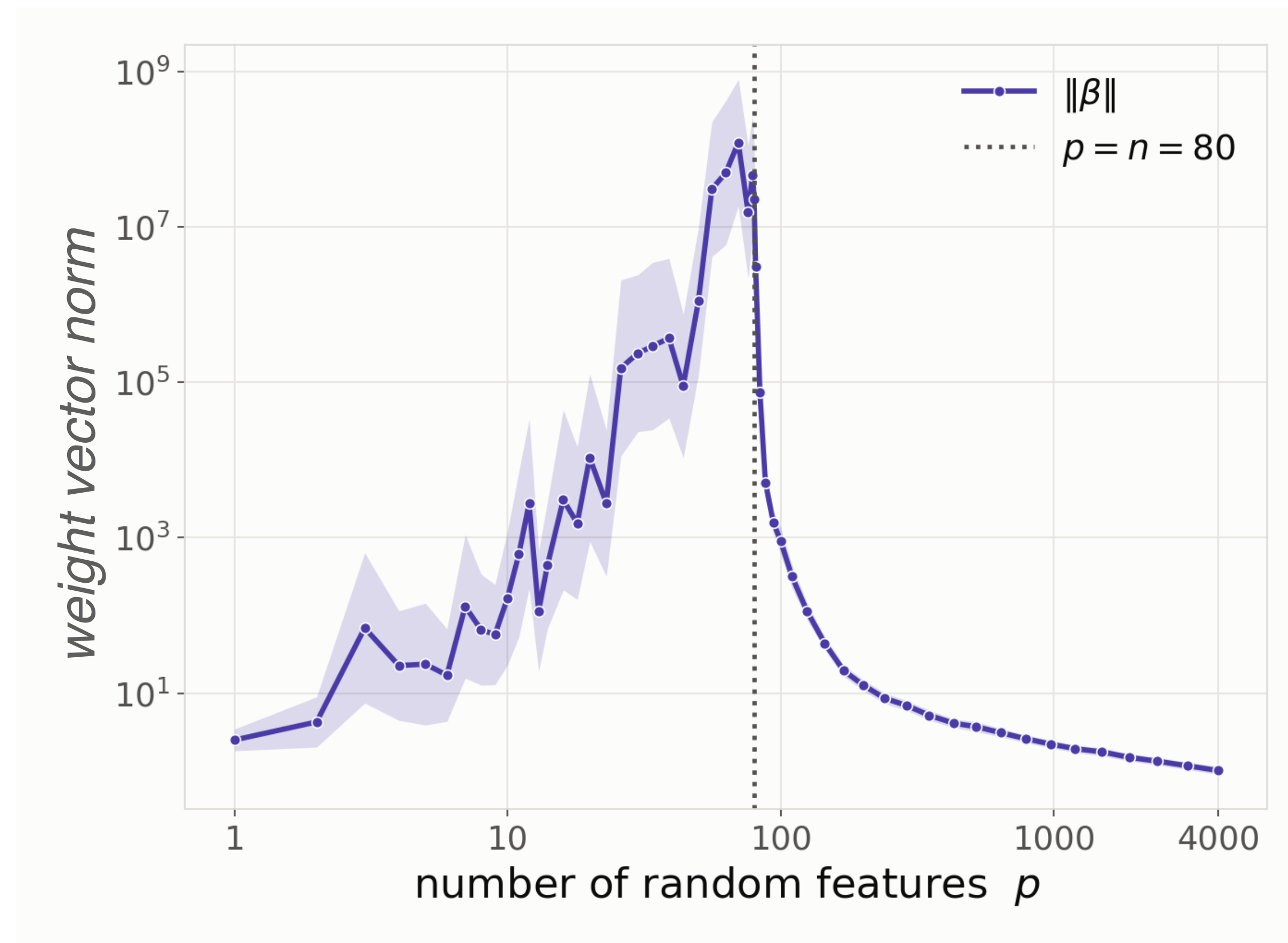
UCI banknote data



- Validation error peaks at # features = # examples, decreases after (note semilog plot)
  - ▶ gets even slightly lower than for best small model
- Training error = hard 0 from 80 features on

# Double descent for linear regression

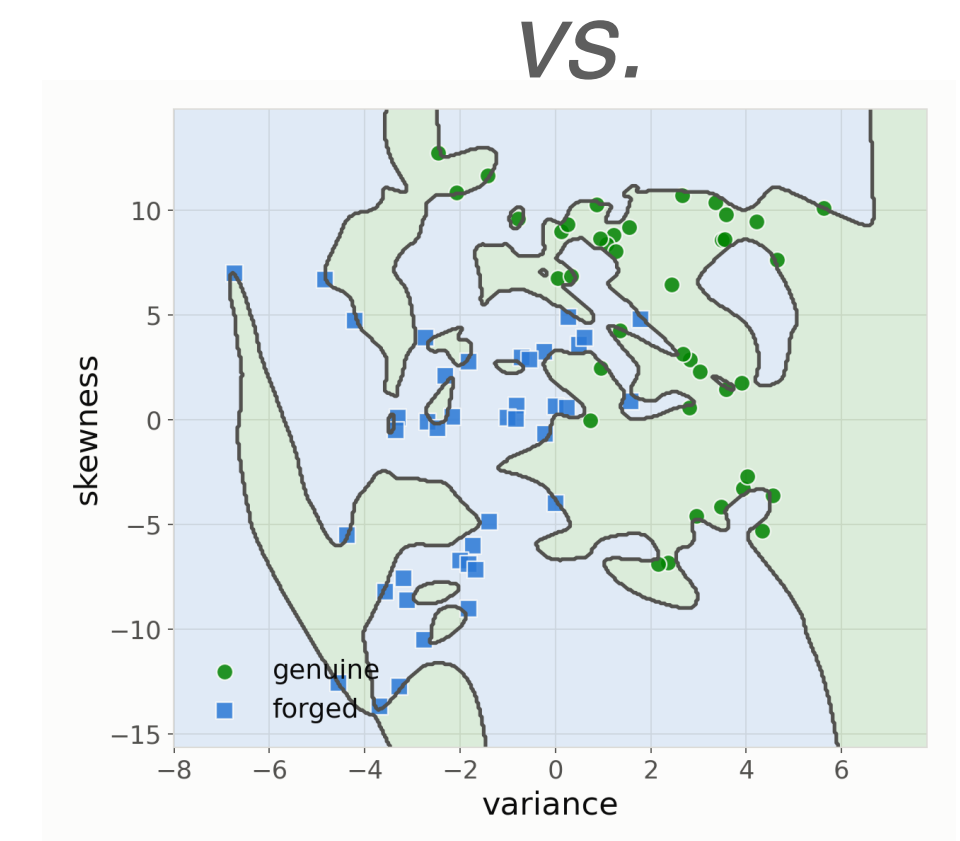
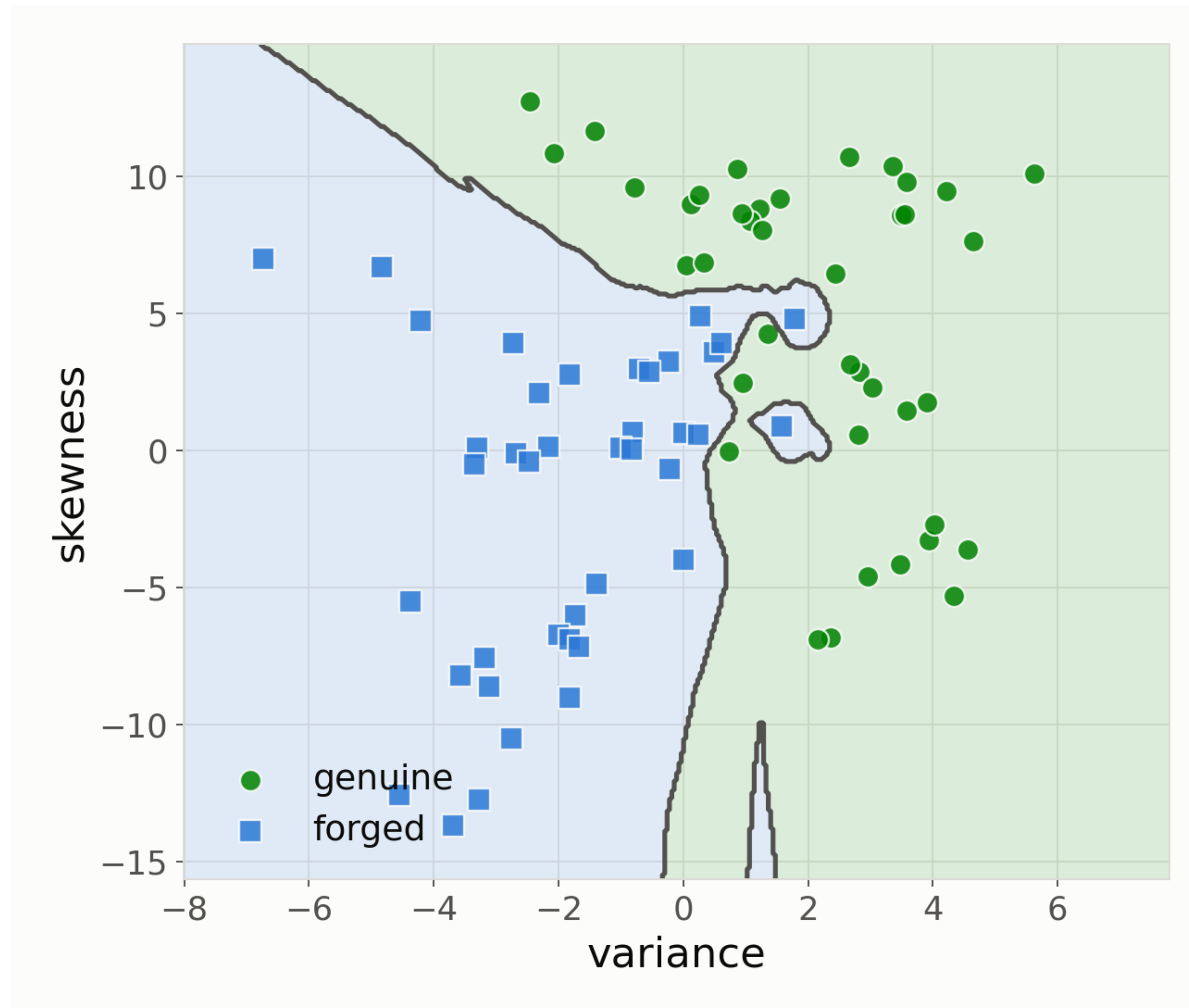
UCI banknote data



- Hint to why we get pattern on previous slide: solution norm spikes at 80 features (note log-log plot)
  - ▶ norm at 4000 features is actually lower than for best small-model solution

# Double descent for linear regression

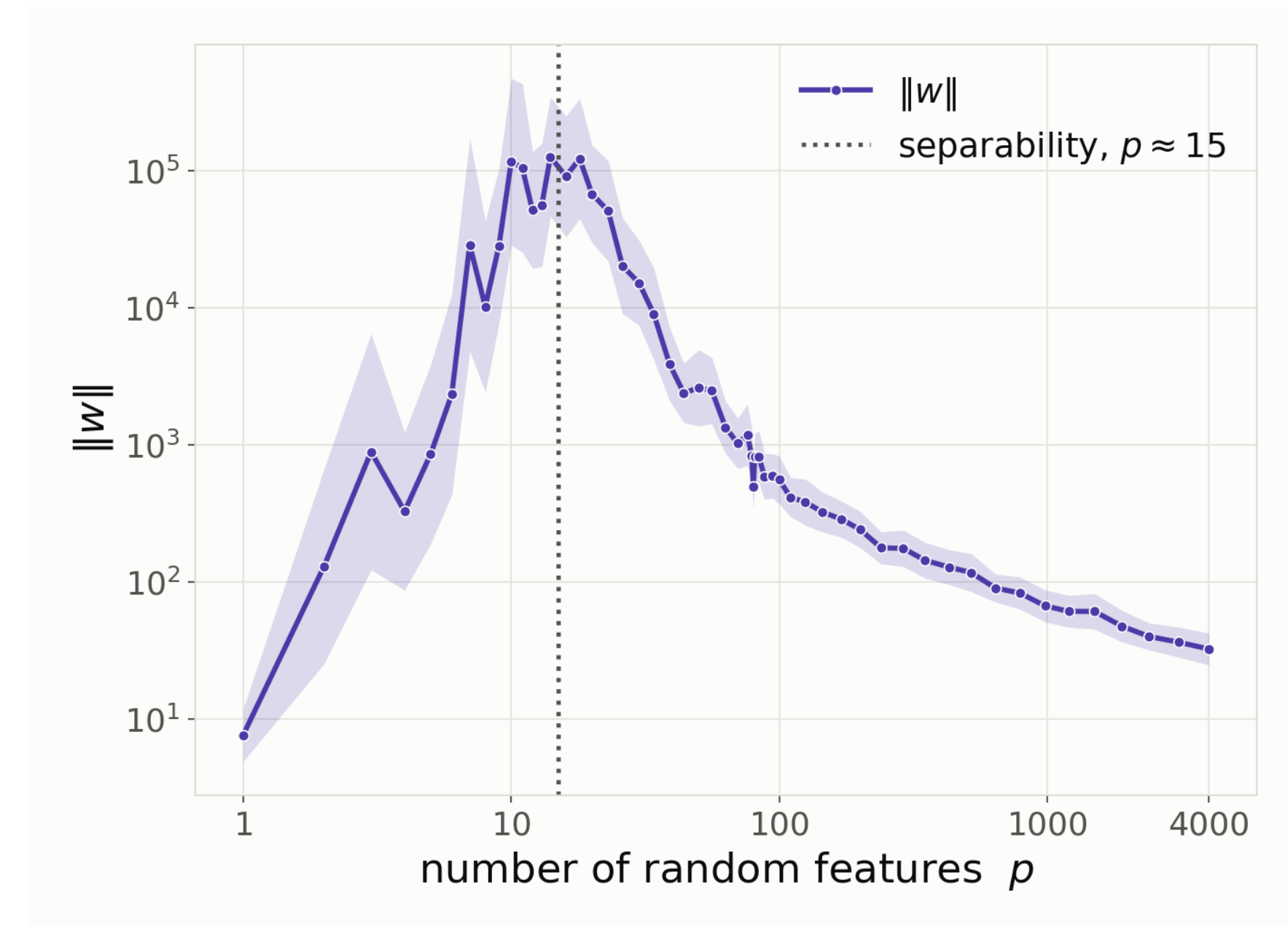
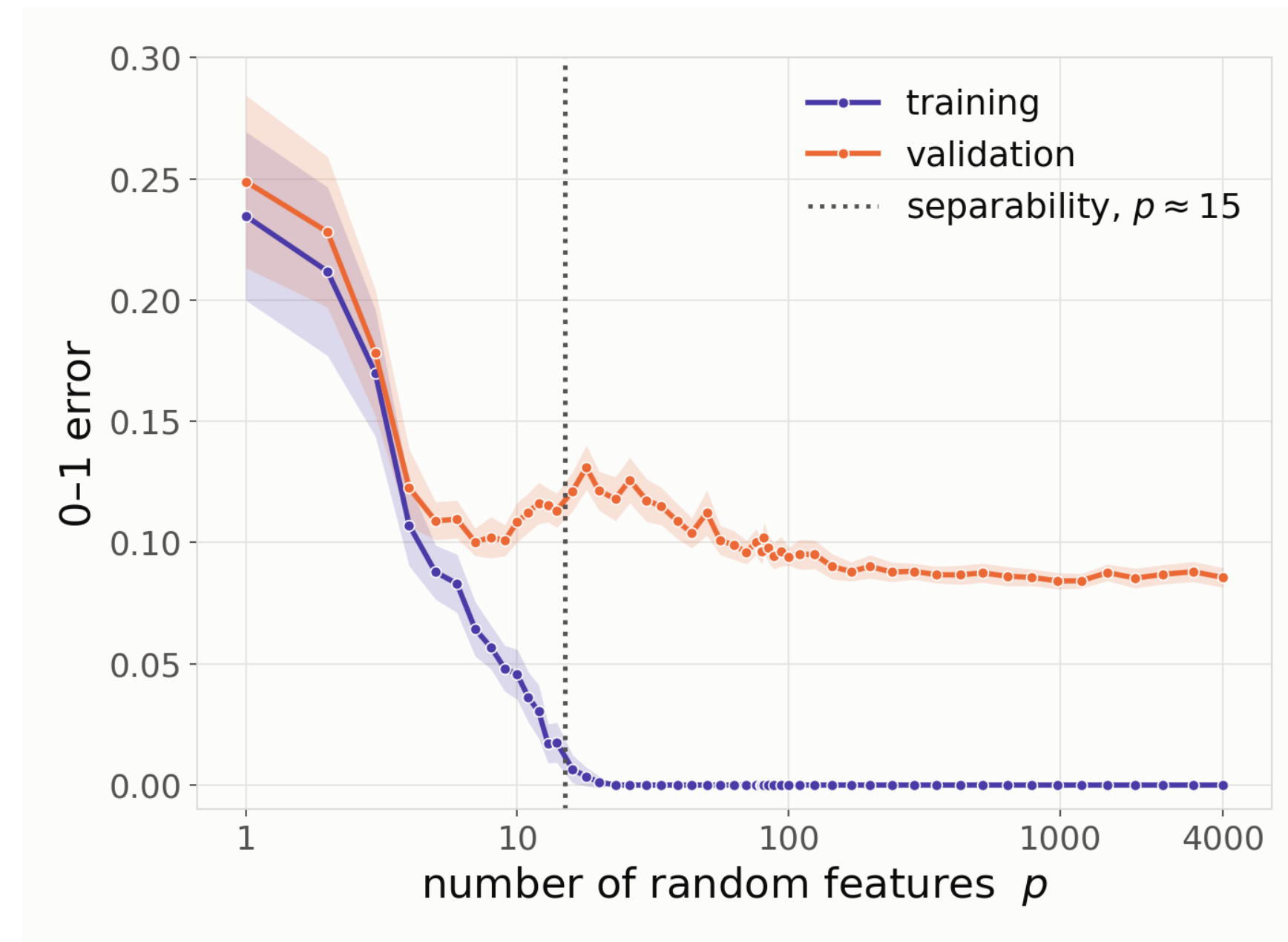
UCI banknote data



- Decision boundary gets simpler again too

# Double descent for logistic regression

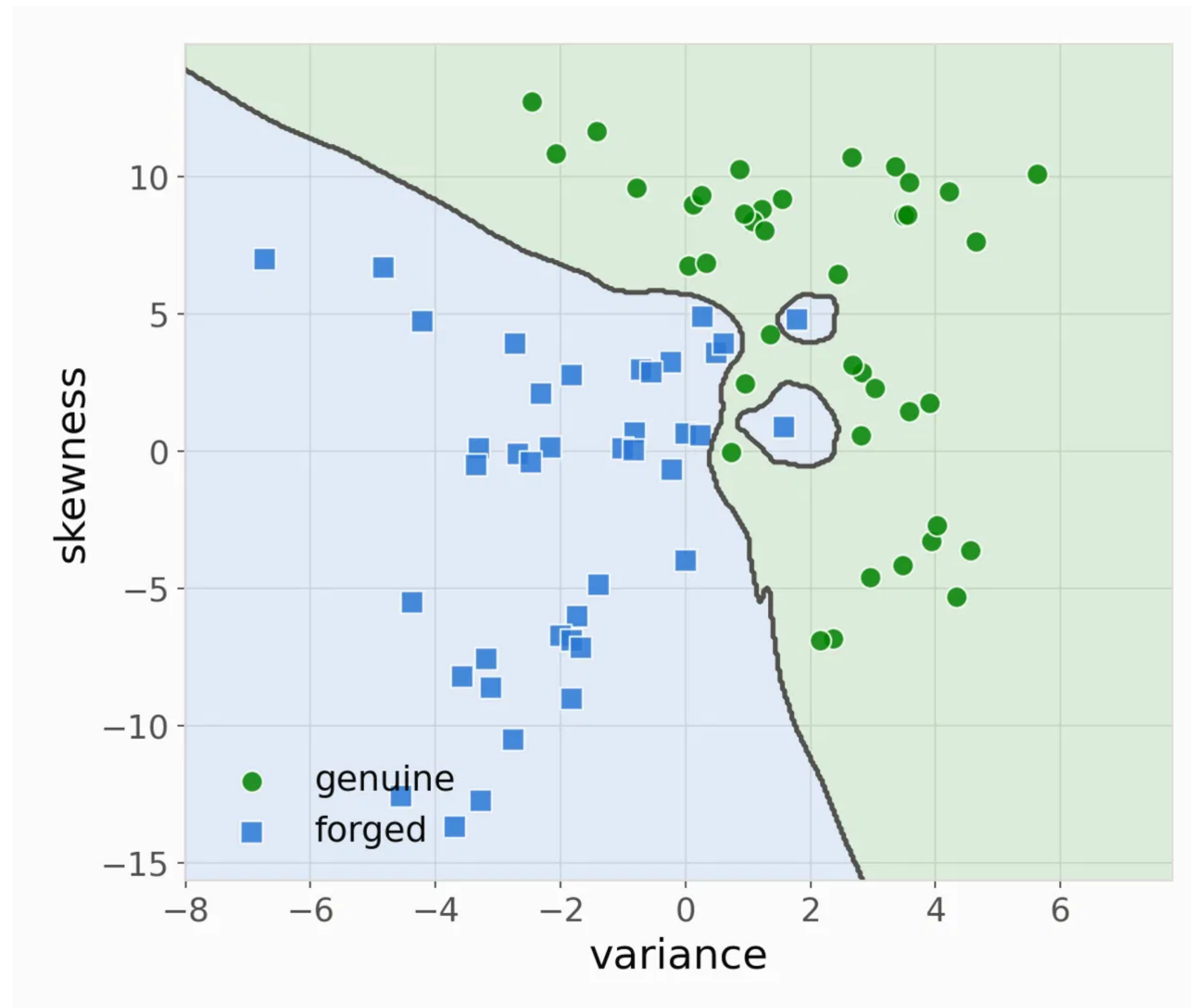
*UCI banknote data*



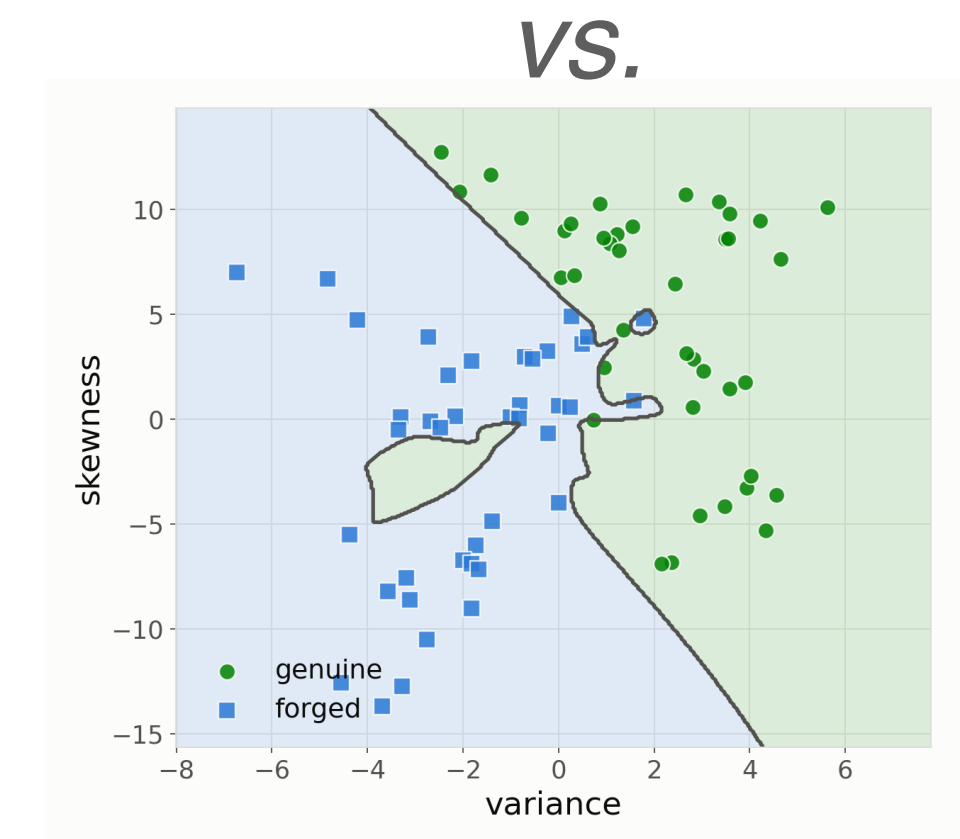
- Similar effect for logistic regression, but
  - ▶ less pronounced peak
  - ▶ shifts left (fewer features)
  - ▶ due to difference between interpolation and separation

# Double descent for logistic regression

UCI banknote data



- Decision boundary



# ***Learning objectives— logistic regression***

- What is the log-likelihood of a Bernoulli distribution? A categorical distribution?
- How do they relate to the sigmoid, softmax, and cross-entropy functions?
- What is a logit or log-odds? What is a logistic regression model? What is its multi-class extension?
- How can we use maximum (conditional) likelihood and SGD to fit logistic regression?

***Learning  
objectives—  
Bayes,  
naive Bayes,  
discriminative,  
constructed  
features***

- What is a Bayes optimal classifier? How does it relate to naive Bayes? To logistic regression?
- What are the naive Bayes classifiers for Gaussian or categorical features?
- How can we use a classifier to learn a distribution?
- How can we learn a nonlinear decision boundary using constructed features?
- How do extra features affect overfitting?
- What is double descent?

# Deep nets



*10-701 Introduction to Machine Learning*  
*Geoff Gordon*

# ***Model = learnable program***

## **Logistic regression**

$z \leftarrow \sum_i w_i x_i$   
return  $\sigma(z)$

## **Perceptron**

$z \leftarrow \sum_i w_i x_i$   
if  $z \geq 0$ :  
  return  $T$   
else:  
  return  $F$

## **Decision tree**

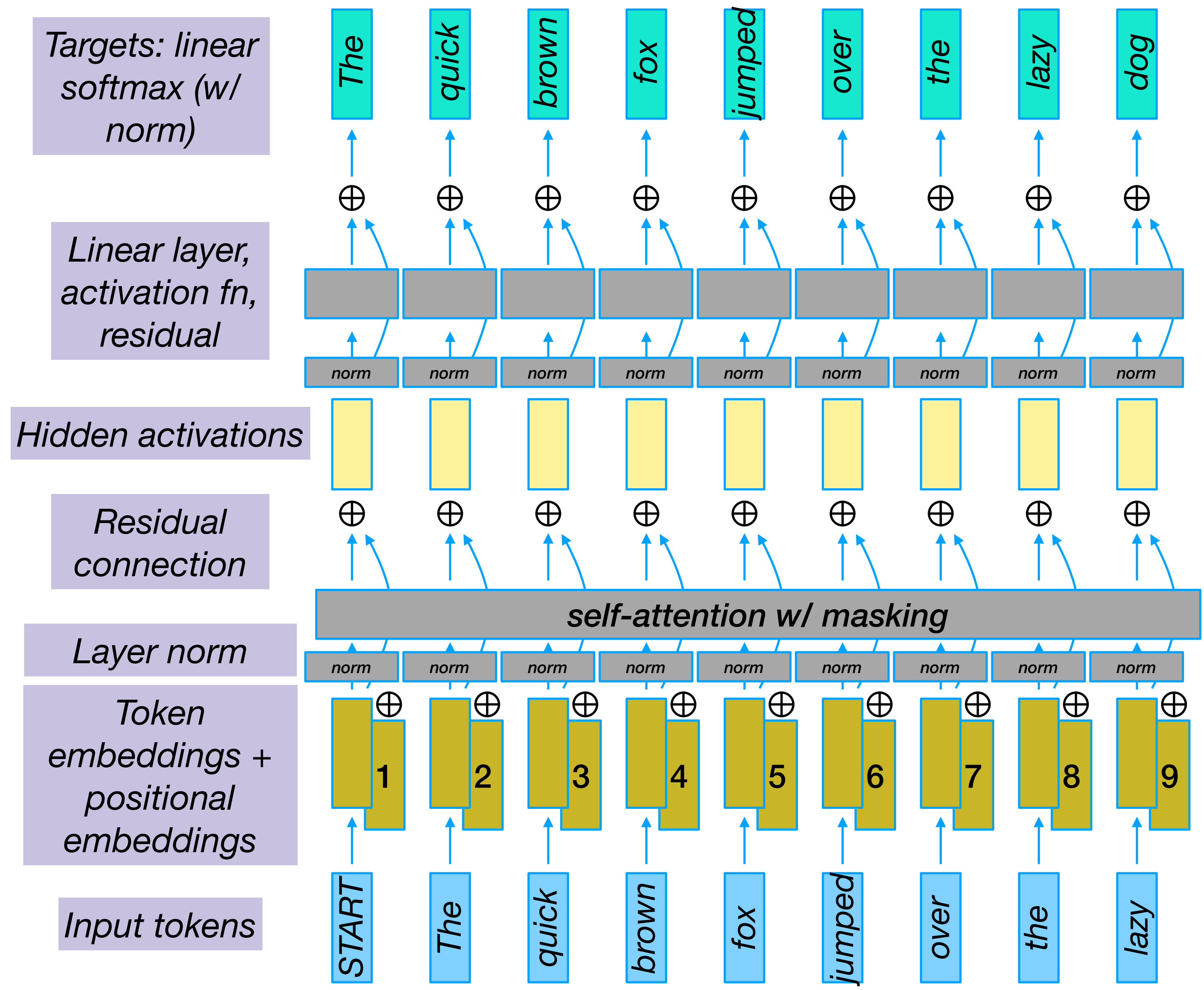
if  $x_1 \leq 3$ :  
  if  $x_2 \leq 5$ :  
    return  $T$   
  else:  
    return  $F$   
else:  
  return  $T$

- Can think of a model as a *decision function (program)* that has *learnable choices*
  - ▶ weight vector  $w$  in perceptron or logistic regression
  - ▶ split axes and thresholds in decision tree

# ***Neural networks***

- Decision functions composed from blocks of
  - ▶ trainable linear functions
  - ▶ fixed (not trainable) nonlinear functions, often chosen from a small list like  $\sigma(z)$ ,  $z/\|z\|$ ,  $[z]_+$ ,  $z_1 \circ z_2$
- Often:
  - ▶ entire function is continuously differentiable at least once wrt parameters (sometimes w/ sparse breakpoints)
  - ▶ trained by SGD or variations
  - ▶ with depth  $\gg 1$
- Includes linear & logistic regression, perceptron as special cases

# Neural networks



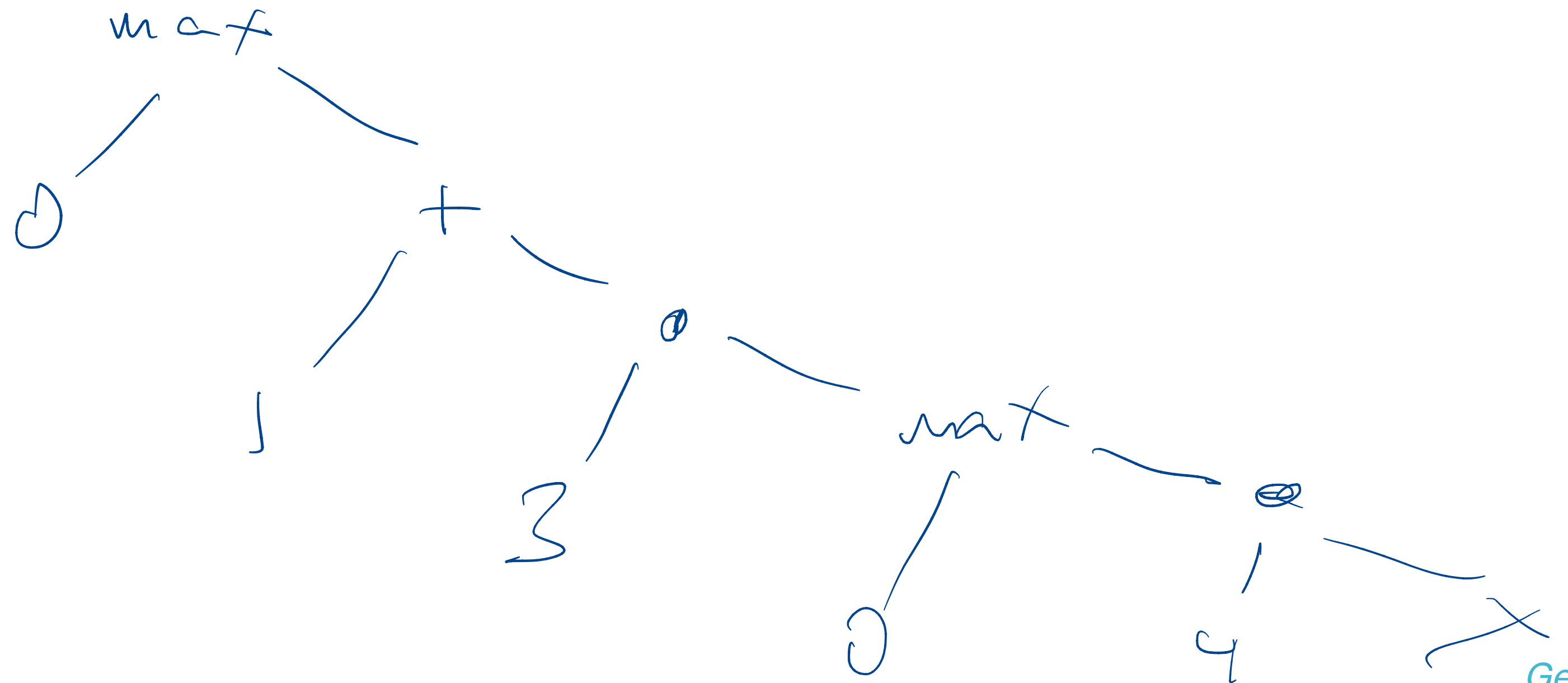
# Deep neural networks

notation:

$$\max(0, z) = [z]_+$$

is **ReLU** or **positive part**,  
one of the most common  
nonlinearities / activation  
functions for deep nets

- Defn: **depth** = minimum number of sequential steps in a computation, given arbitrarily many parallel processors
- In contrast to **width** = max number of processors used, or **work** = total number of processor-steps
- E.g.,
  - ▶  $\max(0, 1 + 3 \cdot \max(0, 4x))$



# ***What is depth?***

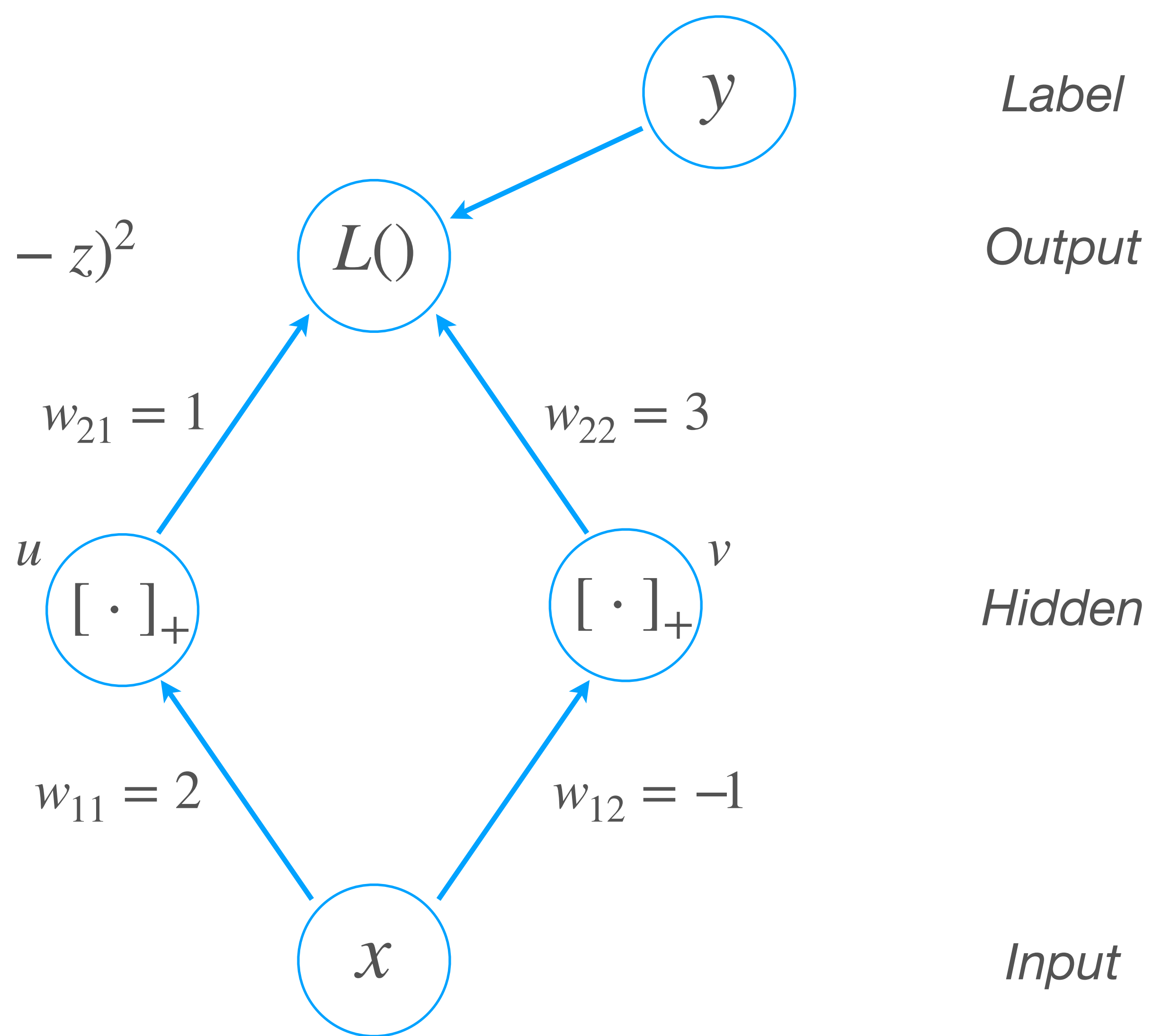
- Width, depth, work are only defined up to constant factors, since we can fiddle with the capability of the processors
  - ▶ convention for deep nets: a processor (a ***neuron***) in one step computes linear function and fixed nonlinearity
  - ▶ a ***layer*** is a group of steps that can be performed in parallel

## Example

a simple neural network predicting label  $y$  from input  $x$  under squared error

$$L(y, z) = \frac{1}{2}(y - z)^2$$

2 layers  
width 2  
work 3

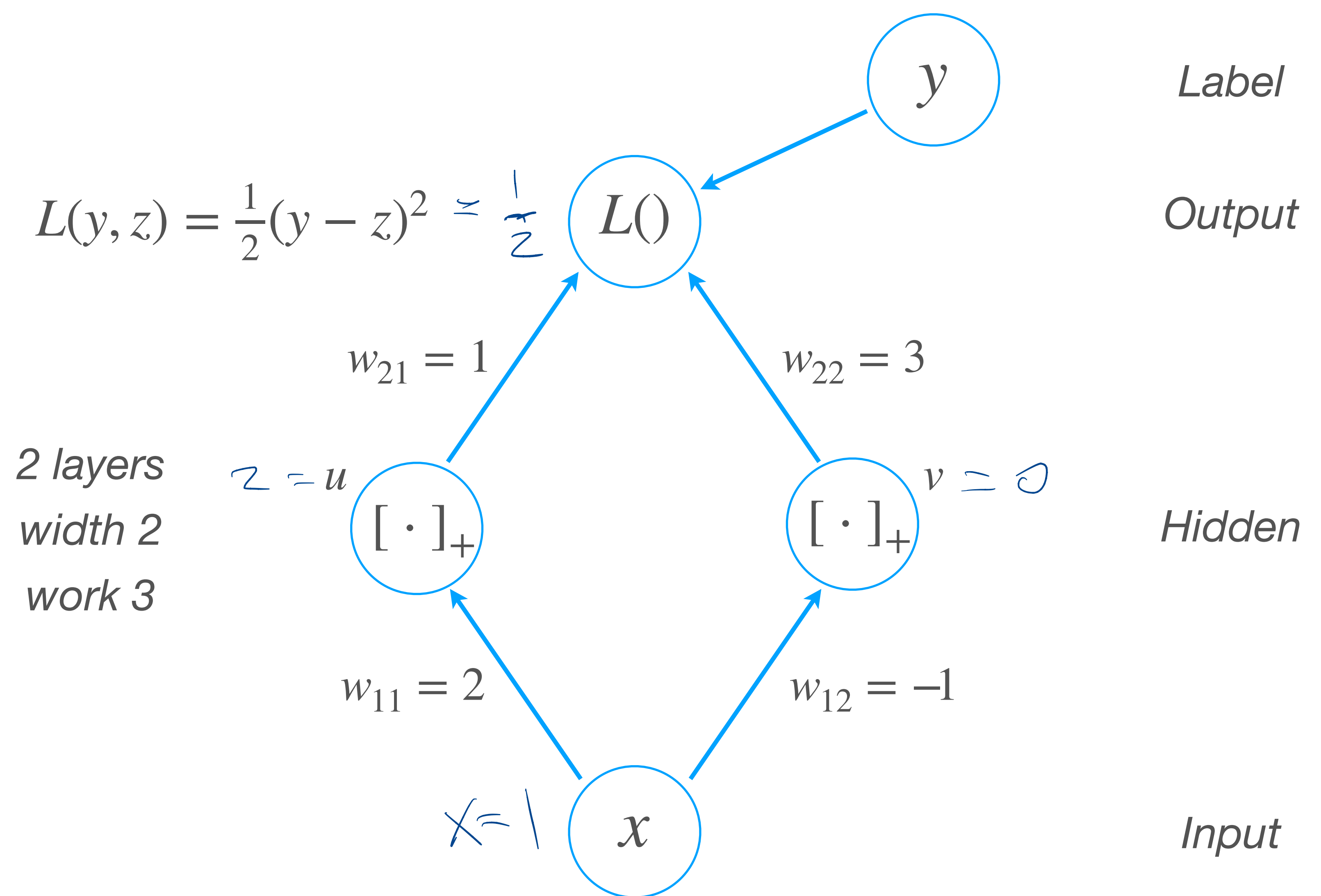


each node computes a real number (an **activation**)  
start at inputs, propagate toward outputs (**forward pass**)  
by convention, weights label edges, but are part of the  
computation of the incident node

try  $x = 1, y = 1$

## Example

a simple neural network predicting label  $y$  from input  $x$  under squared error



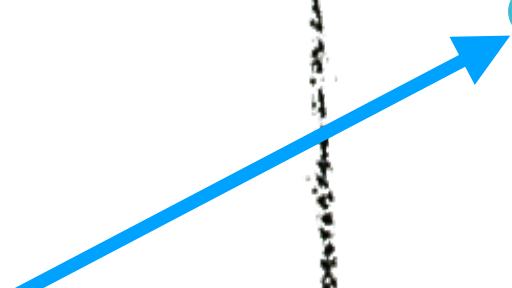
each node computes a real number (an **activation**)  
start at inputs, propagate toward outputs (**forward pass**)  
by convention, weights label edges, but are part of the  
computation of the incident node

# ***How to train your ~~dragon~~ deep net***

- SGD!
- Draw examples  $(x, y)$  from training set
  - ▶ or minibatches of examples
- Run network forward from  $x$  to get a prediction
- Compare to  $y$  to calculate loss function
- Calculate gradient  $g$ : derivative of loss wrt network weights
- Update network weights by subtracting a multiple of  $g$ 
  - ▶ perhaps w/ momentum, Adam, etc.

# ***How to train your ~~dragon~~ deep net***

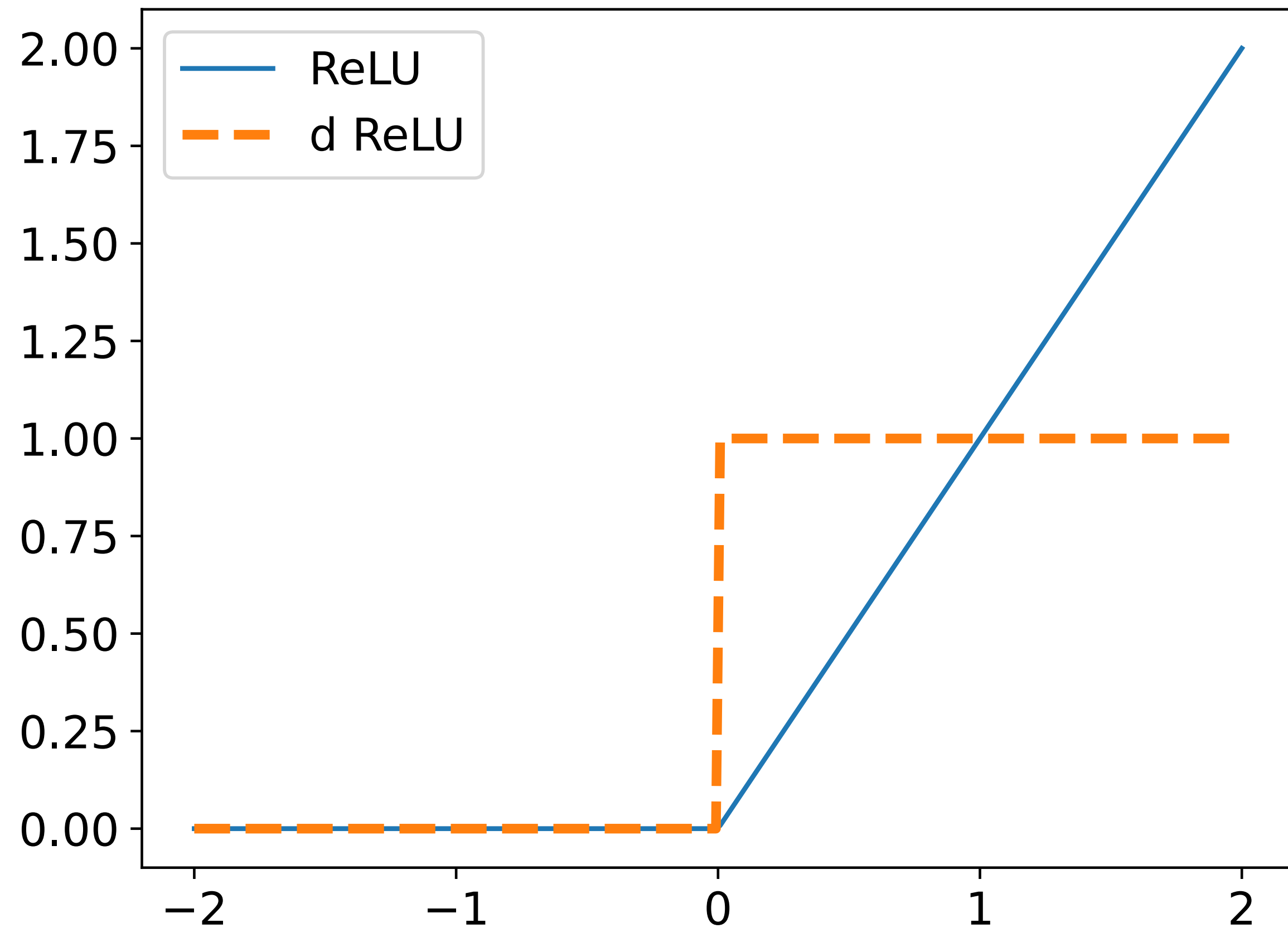
*how might we  
do this?*



- SGD!
- Draw examples  $(x, y)$  from training set
  - ▶ or minibatches of examples
- Run network forward from  $x$  to get a prediction
- Compare to  $y$  to calculate loss function
- Calculate gradient  $g$ : derivative of loss wrt network weights
- Update network weights by subtracting a multiple of  $g$ 
  - ▶ perhaps w/ momentum, Adam, etc.

# Gradient of ReLU

$$[u]_+ = \begin{cases} u & \text{if } u \geq 0 \\ 0 & \text{if } u < 0 \end{cases}$$



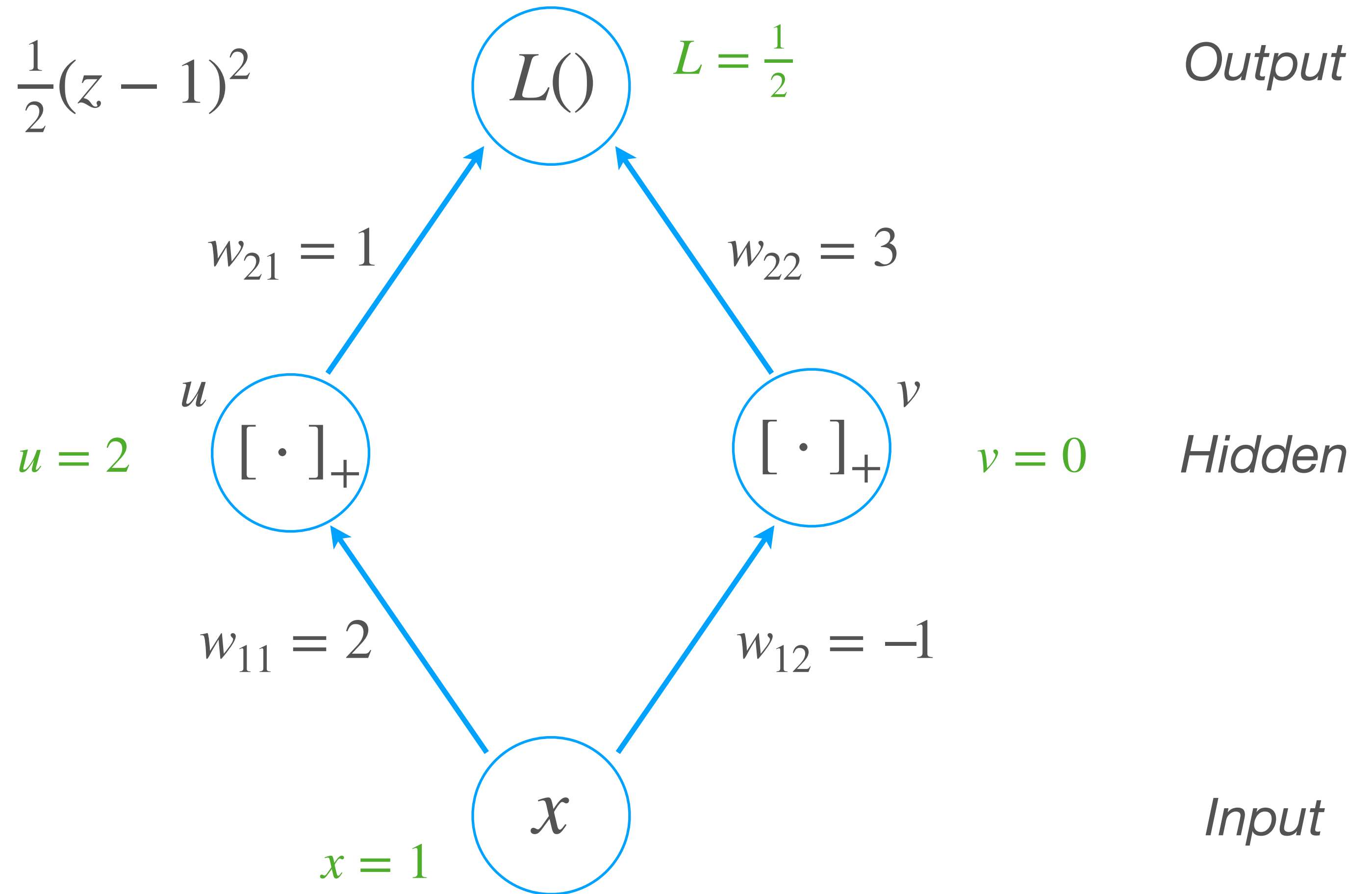
- Derivative of  $[u]_+$ :  $S(u) = \begin{cases} 1 & \text{if } u \geq 0 \\ 0 & \text{if } u < 0 \end{cases}$

# ***Calculate derivatives wrt input $x$ and wrt weights $w_{ij}$***

*we care most about derivatives  
wrt weights, but we can get  
both at once with no extra cost*

*fix  $y = 1$  for simplicity*

$$L(z) = \frac{1}{2}(z - 1)^2$$



$$dL = 2Ax + \dots$$

$$\rightarrow dL = 2Ax$$

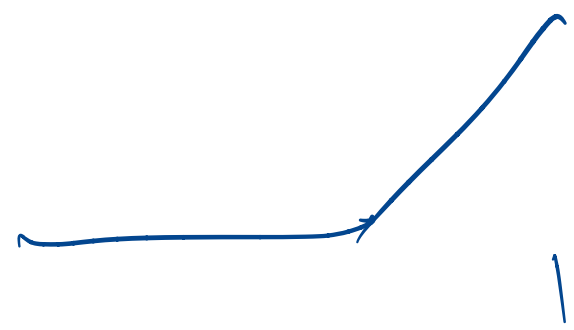
$$dL = 0 dx + \dots$$

*Professor provides*

$$x = 1$$

$$dL = 1 \cdot du + \dots$$

$$x = 1 \quad w_{11} = 2$$



$$du = 1 \cdot d(w_{11}x) = 1 \cdot 2dx$$

$$du = 1 \cdot dw_{11} \cdot x = dw_{11}$$

$$u = [w_{11}x]_+$$

$$dL = 2 \cdot dx + \dots$$

$$dL = dw_{11} + \dots$$

$$dL = 3A v + \dots$$

$$= 0 dx + \dots$$

$$x = 1 \quad w_{12} = -1$$

$$dv = 0 d(w_{12} x) = 0 \cdot (-1) \cdot dx$$

$$dv = 0 \cdot 1 \cdot dw_{12}$$

$$v = [w_{12} x]_+$$

$$dL = 0 dw_{12} + \dots$$

$$dL = \underbrace{(w_{21}u + w_{22}v - 1)}_1 \lambda \underbrace{(w_{21}u + w_{22}v)}_{u=2, v=0} = 2 \quad \begin{matrix} w_{21} = 1 \\ w_{22} = 3 \end{matrix}$$

$$= \underbrace{w_{21}}_{\hookrightarrow 1} du + \underbrace{w_{22}}_{\hookrightarrow 3} dv$$

$$L = \frac{1}{2}(w_{21}u + w_{22}v - 1)^2$$

$$dL = 1 \cdot \left( dw_{21} \cdot \cancel{\frac{1}{2}} + dw_{22} \cdot \cancel{0} \right) = 2dw_{21} + 0dw_{22}$$

# *How did we get the derivative wrt $x$ ?*

- Loop: start above the top (output) node with  $dL = dL$
- Visit nodes in backward order, from output to input (a **backward pass**)
  - ▶ so that we've looked at all successors of a node before we visit it, and know derivative of **overall** loss wrt **current node's** outputs
  - ▶ this is our loop invariant
- At each node, use chain rule to find derivative of loss wrt each **previous node's** outputs (this node's inputs) using known local derivatives for current node

# ***How did we get the derivative wrt $w$ ?***

- At each node, we already computed all of the information we need (activations from forward pass and intermediate gradients from backward pass)
  - ▶ weight gradients can then be computed locally
- Typical setup:
  - ▶ store activations during forward pass, accumulate weight gradients during backward pass
  - ▶ but it's possible to mix cacheing and recomputation to change tradeoff of memory vs. runtime

**Try another example**

$$v = [\omega_{12} x]_+$$

$$dv = 1 \cdot d(\omega_{12} x)$$

